Taming the Performance Unpredictability of the Maximum k-Plex Finding Algorithms by Machine Learning

Anonymous Author(s)

Abstract

A k-plex is a dense structure where every vertex can be disconnected with at most k vertices. Finding a maximum k-plex (MkP) in a big graph is important in many real applications such as community detection and biological network analysis. Numerous MkP algorithms have emerged in recent years featuring a broad range of sophisticated pruning techniques that fall in three categories, those for (1) branching, (2) bounding, and (3) reduction during subgraph exploration. Two recent winners are (1) U-MkP which features effective branching methods, and (2) AltRB which features an alternated reduction-and-bound (abbr. altRB) method, both significantly beating prior works but have not been compared with each other.

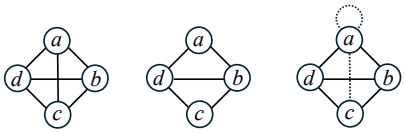
In this paper, we explore how to combine U-MkP's branching technique with the altRB method to achieve the best performance. U-MkP adopts a sparse data representation that favors buffer reuse during backtracking, while AltRB adopts a dense bit encoding representation that favors set operations. There is no clear winner of the data representations so we implement both versions, and they can enable U-MkP's branching technique and the altRB method, giving five algorithm variants. However, the running time of some variants can be orders of magnitude faster than others, but there is no clear winner or loser among these variants and the result vary case by case in terms of the graph and the value of k and is highly unpredictable. We, therefore, explore the use of machine learning to predict the fastest algorithm based on the graph features after an initial reduction pruning as well as the value of k, and extensive experiments show that ..... statement on accuracy based on Naman's results: which models are the best? which models are complementary to each other? ensemble can reach 0 failure? ... Our source code is released at https://???.

ACM Reference Format:

Anonymous Author(s). 2025. Taming the Performance Unpredictability of the Maximum k-Plex Finding Algorithms by Machine Learning. In . ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Finding cohesive subgraphs in a large graph is useful in applications such as finding protein complexes or biologically relevant functional groups [5, 8, 16, 24] and social communities [15, 20]. Clique is a classic notion of cohesive subgraph where every pair of vertices are connected by an edge (see Figure 1(a)). However, communities rarely appear in the form of cliques in real graphs due to various reasons such as the existence of data noise [11, 12, 21, 26]. As a relaxed clique model, k-plex [23] is defined as a graph where every vertex v is adjacent to all but at most k vertices (including v itself). Figure 1(b) shows a 2-plex where every vertex is not adjacent to at most $k = 2$ vertices. For example, a is not connected to $\{a, c\}$ (see dotted edges in Figure 1(c)). It has found extensive applications in the analysis of social networks [23], especially in the community detection [12, 21]. However, mining k-plex structures is NP-hard [6, 19], so existing algorithms rely on branch-and-bound search which runs in exponential time, but utilize effective pruning techniques to make the search process tractable on large graphs.



(a) A Clique (b) A 2-Plex (c) Missing Edges of a

Figure 1: Examples of a Clique and a 2-Plex

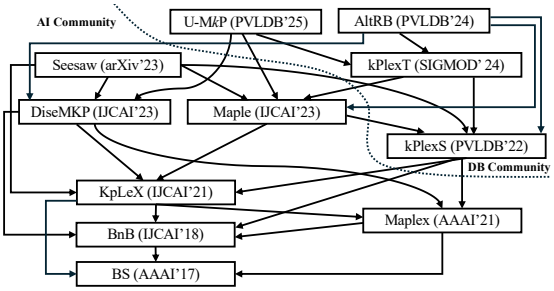


Figure 2: Reported Algorithm Dominance Relationships

In this paper, we study the problem of finding a maximum k-plex (MkP), i.e., only one is returned when there are ties. The MkP algorithm can be extended to find all MkPs using the method introduced in [4]. In recent years, there is a surge of MkP algorithms with new pruning techniques proposed by the DB and AI communities to significantly speed up computation, such as U-MkP [4], AltRB [14], kPlexT [10] Maple [25], Seesaw [28], DiseMKP [17], kPlexS [9], KpLeX [18], Maplex [29], BnB [13] and BS [27]. Figure 2 shows the performance dominance relationships reported by the respective papers to their compared algorithms. Among them, U-MkP and AltRB are winners that significantly beat the MkP other algorithms, but they have not been compared with each other yet.

Specifically, U-MkP [4] is a common framework that unifies the pruning techniques from all the other MkP algorithms except for AltRB [14], which summarizes the techniques into three categories, those for (1) branching, (2) upper bounding, and (3) reduction during subgraph exploration. Through extensive experimental study, a default configuration of the pruning techniques that works generally well has been identified, featuring effective branching methods that depends on the value of k: S-based branching method (we will introduce later) is used when $k \leq 5$ and pivot-based method is

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

used when $k > 5$. AltRB [14], on the other hand, features an alternated reduction-and-bound (abbr. altRB) method that significantly reduces the search space. A key goal of this work is to combine U-MkP's branching techniques with the altRB method to further improve the performance of MkP finding.

Interestingly, our analysis reveals that U-MkP and AltRB employ distinct data structures for maintaining the explored and candidate vertex sets, exhibiting unique tradeoffs between the *ability of incremental updates* and the *computational overhead of set operations*. It is difficult to find simple rules to predict which data representation wins a priori, as results vary significantly across different problem instances. To harness these complementary strengths, we implement: (1) U-MkP's two branching techniques within AltRB, and (2) the altRB method within the U-MkP framework. Combined with the original U-MkP and AltRB, this yields five competitive algorithms for MkP finding, providing a portfolio of high-performance candidates to choose from for execution at the runtime.

While certain algorithm variants demonstrate orders-of-magnitude speed improvements over others, no single variant consistently outperforms across all scenarios. The relative performance depends intricately on both graph characteristics and the parameter k, exhibiting high unpredictability. To address this, we develop a machine learning approach to predict the fastest algorithm using graph features (after initial reduction pruning) and the value of k.

This prediction is essentially a five-class classification problem, but requires specialized treatment in the machine learning workflow to optimize algorithm selection. Specifically, in some problem instances, multiple winner candidates demonstrate statistically indistinguishable performance (e.g., $\leq$ 5% faster or simply a tie), requiring a model to accommodate all optimal algorithm options (rather than to force a single prediction) during training and evaluation. Given a problem instance (i.e., a graph plus a parameter k), the focus should be to avoid selecting a poor algorithm option that runs much slower (e.g., by orders of magnitude) than the fastest.

For training, we consider a *time-discrepancy-based sample weighting scheme*, which gives a training sample $\mathbf{x}$ a higher weight $w(\mathbf{x})$ if the running time of different algorithm candidates varies a lot. Note that making the correct prediction for such a sample is crucial since a wrong prediction would lead to a much longer running time, while in contrast, the algorithm selection is not important if all candidates run for more or less the same time. We also consider a *tie-aware weight splitting approach*, where given a training sample $\mathbf{x}$, if k algorithms are all the fastest with a tie (!!! to Naman: not just handling exact ties but also accommodating/tolerating small differences for replicating, e.g., if time gap is within 5%), then we create k instances of this sample with each instance replicating $\mathbf{x}$'s input features but using a different algorithm label from the k algorithms. Each instance has weight $w(\mathbf{x})/k$ so that the weight of all instances sum up to be $w(\mathbf{x})$. Finally, we adopt *(dense) probabilistic categorical cross-entropy loss* for supervision whenever applicable, where "soft" probabilistic labels are used so that algorithms with close running times will have similar probabilities of being selected.

For evaluation, we define metrics such as accuracy, precision and recall to accommodate ties, i.e., a prediction is considered correct as long as any of the fastest algorithms is selected. (!!! to Naman: to generalize tie to allow 5% tolerance of time to be considered correct) We also define time-based metrics that indicate the relative increase of running time when using the model-predicted algorithm instead of the fastest algorithm (assuming there is an oracle predictor that does not make mistakes). ... say sth about failure cases to visual examination, and ensemble ...

To curate a dataset for model training and evaluation, we collecte a total of 2741 graphs from three sources: SNAP [3], KONECT [1] and Network Repository [2], which include both real and synthetic graphs. As a preprocessing step before subgraph exploration, kPlexS [9] proposes to use a heuristic algorithm to find a large k-plex P as well as an upper bound ub on the size of any MkP in G, by running the peeling algorithm in linear time. If $|P| = ub$, then P is already an MkP and can be directly returned; otherwise (2) we can still prune those search branches that cannot lead to a larger k-plex with size at least $(|P| + 1)$ during subgraph exploration. This preprocessing step is adopted by the latest algorithms including U-MkP [4] and AltRB [14], and this work also adopts it. Among the 2741 graphs, we exclude 2426 graphs since either the fast preprocessing directly finds an MkP on them, or all five candidate algorithms run for more than 1800 seconds (which is our timeout threshold to avoid long-running jobs) for all tested values of k. Among the remaining graphs, an MkP can be found on 198 of them in less than a second by all algorithms for all values of k, so we also exclude them, leaving only 117 graphs in our curated dataset that are interesting for our machine learning study.

Each combination of dataset and k gives one sample for machine learning, so our dataset is not large and we only consider conventional statistical machine learning models and small neural network models. We remark that our subgraph feature extraction does not incur extra cost since all our graph statistics are straightforward to obtain, including the sizes of k-cores which is a byproduct of the preprocessing step that runs the peeling algorithm. Moreover, deploying a trained model for algorithm selection given a new graph takes negligible time since we use small models. ... todo: comment on model performance like in abstract...

The main contributions of this paper are summarized as follows:

- We explore the two winner MkP algorithms U-MkP [4] and AltRB [14], and propose a unified algorithm framework to combine their pruning techniques. Using this algorithm framework, we screened thousands of public graphs to curate a dataset of graphs interesting for our learning problem.
- We survey the difference of data structures for vertex sets used by U-MkP and AltRB during subgraph exploration and their pros and cons, and propose five competitive algorithm candidates decided by the data structure and adopted pruning techniques for learning-based selection.
- For machine learning, we propose techniques to put larger weights on samples where the selection decision is critical during training, and techniques to accommodate winning algorithms with ties during both training and evaluation.
- Extensive experiments are conducted which ... todo: comment on model performance like in abstract...

The rest of this paper is summarized as follows. Section 2 reviews the set enumeration procedure used by MkP algorithms, and surveys recent advances in MkP techniques. Then, Section 3 presents our unified MkP framework, detailing search-space pruning strategies unique to U-MkP and AltRB. Section 4 describes the distinct set

representation methods employed by U-MkP and AltRB. Section 5 presents our five competitive algorithm variants generated by combining different pruning techniques and set representations, and proposes our learning-based algorithm selection method. Finally, Section 6 reports experiments and Section 7 concludes this paper.

2 Preliminaries and Related Work

Notations. We consider an undirected and unweighted simple graph $G = (V, E)$, where V is the vertex set, and E is edge set. The degree of a vertex v is denoted by $d_G(v) = |N_G(v)|$. We also define the concept of *non-neighbor*: a vertex u is a non-neighbor of v in G if $(u, v) \notin E$. Accordingly, the set of non-neighbors of v is denoted by $\overline{N_G}(v) = V - N_G(v)$, and we denote its cardinality by $\overline{d_G}(v) = |\overline{N_G}(v)|$. Given a vertex subset $S \subseteq V$, we denote by $G[S] = (S, E[S])$ the subgraph of G induced by S, where $E[S] = \{(u, v) \in E \mid u, v \in S\}$. We simplify the notation $N_{G[S]}(v)$ to $N_S(v)$, and define other notations such as $N_S(v)$, $\overline{N_S}(v)$ and $d_S(v)$ in a similar manner. For an arbitrary graph g, $V(g)$ and $E(g)$ denote the vertex set and edge set of g, respectively. The diameter of G, denoted by $\Delta(G)$ is the shortest-path distance of the farthest pair of vertices in G, measured by the number of hops.

Problem Definition. We next define k-plex and MkP.

Definition 1. *(k-Plex) A graph g is a k-plex if every vertex $v \in V(g)$ has at least $|V(g)| - k$ neighbors in g, i.e., $d_g(v) \geq |V(g)| - k$. Equivalently, g is a k-plex if every vertex $v \in V(g)$ has at most k non-neighbors in g (including v itself as a non-neighbor), i.e., $\overline{d_g}(v) \leq k$.*

Definition 2. *(Maximum k-Plex Finding) Given a graph G, the maximum k-plex finding problem finds a largest vertex set $P \subseteq V$ such that the subgraph $G[P]$ induced by P is a k-plex.*

Hereditariness and Diameter of k-Plex. Note that k-plex satisfies the hereditary property which says that: any induced subgraph (denoted by g') of a k-plex g is also a k-plex, since a vertex in g' cannot miss more neighbors than those already missed in g.

Moreover, as proved by [27], the diameter of k-plexes with a reasonably large size (which is usually the case for MkPs) is bounded:

Theorem 1. *For a k-plex P and any integer $c \geq 2$, if $|P| > 2k - c$, then $\Delta(P) \leq c$ [27].*

Most works [9, 25] only consider the case when $c = 2$ by assuming $|P| \geq 2k - 1$, which gives the following corollary:

Theorem 2. *Given a k-plex P, if $|P| \geq 2k - 1$, then $\Delta(P) \leq 2$.*

The assumption $|P| \geq 2k - 1$ is reasonable for a maximum k-plex P, since natural communities that our MkP problems aim to discover are connected, and $2k - 1$ is relatively small. For example, when $k = 5$, we only require $|P| \geq 9$. Note that a k-plex with $|P| = 2k - 2$ may be disconnected (e.g., formed by two disjoint $(k - 1)$-cliques).

Subgraph Exploration by Branch and Bound. All existing MkP algorithms follow (but are variants of) the branch-and-bound search framework shown in Algorithm 1, where S is the set of vertices already included into the current subgraph, and R is the set of candidate vertices yet to be added to S to form a larger subgraph that can become an MkP. Specifically, given a vertex-set pair $\langle S, R\rangle$ in graph g, Line 2 first calls a function reduce_and_prune(S, R, g) to apply reduction and upper-bounding rules, which may decide

Algorithm 1: Basic Branch-and-Bound Search

```
1 function BB_basic(S, R, g)    # The basic BB(.) variant
2     if (reduce_and_prune(S, R, g) = true) then return
3     for each v ∈ R do
4         BB_basic(S ∪ {v}, R − {v}, g)
5         R ← R − {v}
```

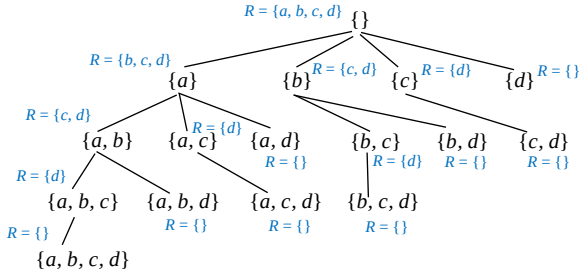


Figure 3: Set-Enumeration Search Tree

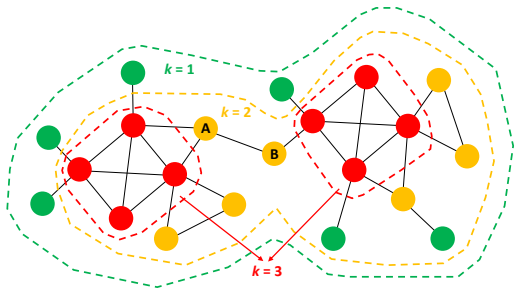


Figure 4: Illustration of k-Cores

that the entire search branch to extend S is unpromising so that *true* is returned, in which case Line 2 returns directly to terminate the extension of S. We will discuss this function in more detail in Section 3. If the branch is not pruned, Line 3 then takes the next candidate $v \in R$, and split the search space into two cases: (1) v is in the MkP to find, in which case Line 4 further extends $\langle S \cup \{v\}, R - \{v\}\rangle$ by recursion; (2) v is not in the MkP to find, in which case Line 5 removes v so that it will not appear in future iterations of the for-loop in Line 3. Figure 3 illustrates the search process of Algorithm 1 on a toy graph with four vertices a, b, c and d, which corresponds to a set-enumeration search tree where each node denotes S and we also annotate its corresponding R near the node (assuming that no node is pruned, and that vertices in R are always ordered with $a < b < c < d$). We can see that the search space is perfectly partitioned without redundancy. Note that calling Algorithm 1 on S basically grows the set-enumeration subtree under node S, which we denote by T_S.

Degeneracy Ordering. The k-core of a graph G is its largest induced subgraph with minimum (vertex) degree k. The degeneracy of G, denoted by $D(G)$, is the largest value of k for which a k-core exists in G. For example, Figure 4 shows the 1-core, 2-core and 3-core of a graph. Specifically, the 2-core contains all yellow and red vertices in the yellow dashed contour, since it is the largest induced subgraph where every vertex has degree ≥ 2, as any green vertex has degree 1. Note that even though vertex A has degree 3,

Table 1: MkP Algorithms and Their Pruning Techniques [4]

Algorithm	Upper Bounding (UB)					Branching					Reduction						
	S-Based	R-Based	SR-Based	UBR1	UBR2	Binary	Pivot	S-Based	R-Based	SR-Based	Two-Hop	RR1	RR2	RR3	CTCP	BR1	BR2
U-MkP (Our Work)	✓	✓	✓		✓		✓	✓	✓	✓	✓[g]	✓	✓	✓	✓[T]	✓	✓
kPlexT (SIGMOD'24)					✓	✓					✓[g]	✓	✓	✓	✓[T]	✓	✓
Maple (IJCAI'23)				✓			✓				✓[g]	✓	✓	✓	✓[+]	✓	✓
Seesaw (arXiv'23)	✓[+]	✓[+]	✓	—	—	—	—	—	—	—	—	—	—	—	—	—	—
DiseMKP (IJCAI'23)	✓[+]							✓[+]				✓	✓	✓	✓[T]		
kPlexS (PVLDB'22)				✓		✓					✓[g]	✓	✓	✓	✓[+]	✓	✓
KpLeX (IJCAI'21)	✓							✓				✓	✓	✓			
Maplex (AAAI'21)		✓				✓					✓	✓	✓	✓	✓[T]		
BnB (IJCAI'18)						✓						✓	✓	✓			
BS (AAAI'17)							✓				✓	✓	✓	✓			✓

* Note: ✓[g]: Two-hop graph constructed; ✓[+]: Improved version; ✓[T]: Top-level only
—: Seesaw only proposes UB rules, no open-source code;

it is not in 3-core since its neighbor B has degree 2 so cannot be in 3-core, hence A has at most 2 neighbors in 3-core. The operation of k-core decomposition finds the core number of every $v \in V$, denoted by $core(v)$, which is the largest value of k that v belongs to a k-core. For example, $core(A) = 2$ in Figure 4 since A is in 2-core but not 3-core. The process of k-core decomposition [22] can be computed by a peeling algorithm that repeatedly removes a vertex with the minimum current degree at a time [7]. For the graph G in Figure 4, green vertices will first be removed, followed by yellow ones, and finally the red ones, after which no vertex is remaining so we have $D(G) = 3$. Accordingly, the algorithm also generates the degeneracy ordering of vertices in G where green vertices go first, followed by yellow ones and then red ones. Note that $D(G)$ is a small value in real graphs, since a high-degree vertex tends to appear later in the degeneracy ordering, so many of its neighbors have already been removed by peeling.

Related Work on MkP Algorithms. Recall from Figure 2 on Page 1 that many MkP algorithms have been proposed in recent years. Earlier works are mainly from the AI community (since it is a search problem) including Maple [25], Seesaw [28], DiseMKP [17], KpLeX [18], Maplex [29], BnB [13] and BS [27]; while more recent works are from the DB community including U-MkP [4], AltRB [14], kPlexT [10] and kPlexS [9]. These MkP algorithms have proposed many sophisticated pruning techniques during subgraph exploration that fall in three categories, those for (1) branching, (2) bounding, and (3) reduction. The work of U-MkP [4] unifies all techniques into its framework except for new ones proposed by AltRB [14], as summarized in Table 1 (directly from [4]), and please refer to [4] for the detailed statement and proof of each technique. In this paper, we focus on describing those different techniques adopted by U-MkP's default configuration (that is tuned to work well in general) and AltRB that really matter for performance (note that some pruning techniques turn out to have negligible impact on real performance), while skipping the common techniques enabled by all the competitive algorithms, such as UBR2 in Table 1 that is initially proposed by kPlexT [10] and turns out to be so effective that subsequent works U-MkP and AltRB adopt it by default.

To achieve good performance consistently, U-MkP [4] uses S-based branching method when $k \leq 5$ and pivot-based method when $k > 5$. AltRB [14] proposes an alternated reduction-and-bound (abbr. altRB) method that significantly reduces the search space and not covered by U-MkP. Besides these effective techniques, U-MkP and AltRB share a very similar algorithm framework making it easy to unify them, except that (1) AltRB replaces CTCP with a variant reduction technique called CF-CTCP, and (2) it replaces the heuristic algorithm originally proposed by kPlexS [9] to find an initial large k-plex during the preprocessing step with a different heuristic algorithm called KPHeuris. These changes do not show prominent performance improvement on most graphs as shown in the ablation study results from Tables 12-15 in the appendix of [14].

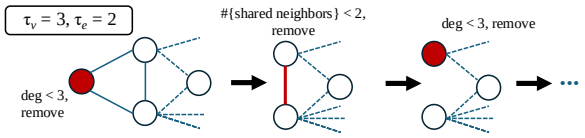


Figure 5: Illustration of CTCP

3 MkP Algorithm Framework and Techniques

In this section, we present the algorithm frameworks of U-MkP [4] and AltRB [14], focusing on the branching methods of U-MkP and the alternated reduction-and-bound (abbr. altRB) method of AltRB.

3.1 A Unified Algorithm Framework

Initial Graph Shrinking & CTCP. Algorithm 2 shows the pseudocode to create top-level search branches $T_{\{v_i\}}$ as ***adopted by both U-MkP and AltRB***, where Line 4 first shrinks the input graph G based on the parameter k. Let P be the current largest k-plex found, then to find a larger k-plex with size at least $|P|+1$, we only need to consider those vertices with degree at least $\tau_v = (|P|+1-k)$ (otherwise, the vertex will miss more than k edges in the larger k-plex). Line 4 uses the CTCP (Core-Truss Co-Pruning) reduction method proposed in kPlexS [9] that is a tighter version: we only need to consider those vertices with degree at least $\tau_v = (|P|+1-k)$ and those edges (u, v) where u and v share at least $\tau_e = (|P|+1-2k)$ common neighbors (see [4] for the details). Figure 5 shows an illustration where vertices and edges below thresholds are alternately pruned. AltRB uses a variant called CF-CTCP that further reduces the redundant computations in CTCP. Since subgraph exploration is conducted on the shrunk G after Line 4, ***our machine learning model uses input features extracted from the shrunk graph rather than the original input graph that is much larger.***

Besides shrinking the initial graph G in Line 4, CTCP is also called in Line 12 (right after Line 11 removes v_i from G), to shrink G further for use by future iterations, since new edges (e.g., v_i's adjacent ones) may have $d_G(u, v)$ reduced below τ_e. By passing $\{v\}$ as the second argument, CTCP can propagate pruning from v and those newly updated edges with $d_G(u, v) < \tau_e$ to minimize its graph update footprint using the method described in [9].

Top-Level Branching & Top-Level Subgraphs. In Algorithm 2, we assume there are two globally maintained variables accessible any time during subgraph exploration: (1) P: the current largest k-plex found, and (2) ub: size upper bound of an MkP. These two variables are initialized in Line 2 using a heuristic algorithm that finds a large initial P as a preprocessing step; P will also be updated during subgraph exploration whenever a larger k-plex is found, so that if we know that a branch cannot generate a k-plex larger than $|P|$, it can be pruned without exploration. If $|P|$ equals ub, then the initial P is already an MkP and is thus returned in Line 3. Otherwise, Line 4 then further prunes G using CTCP.

Algorithm 2: Top-Level Branching ($c = 2$)

Input: A graph $G = (V, E)$ and an integer $k \geq 2$
Output: A maximum k-plex in G

```
1  function top_branching(G)
2      P, ub ← find_init(G)    # global variables
3      if |P| ≥ ub then return P
4      CTCP(G, ∅, |P| + 1 − k, |P| + 1 − 2k)
5      Sort [v_1, ..., v_n] following degeneracy ordering of G
6      for i = 1 to |V| do
7          H_1 ← N(v_i) ∩ {v_{i+1}, ..., v_n}
8          H_2 ← N(H_1) ∩ {v_{i+1}, ..., v_n}
9          g ← the subgraph of G induced by ({v_i} ∪ H_1 ∪ H_2)
10         BB({v_i}, V(g) − {v_i}, g)    # call a BB variant
11         Remove v_i from G
12         CTCP(G, {v_i}, |P| + 1 − k, |P| + 1 − 2k)
13     return P
```

Recall from Theorem 1 that reasonably large k-plexes ($|P| > 2k - c$) have diameter $\Delta(P) \leq c$. In other words, for the top-level nodes v_i ($v_i = a, b, c, d$) in Figure 3, the branch $T_{\{v_i\}}$ under it only needs to consider a subgraph of G, denoted by $g_i = (V_i, E_i)$, where vertices are within c hops from v_i. We call g_i as the ***top-level subgraph*** of $T_{\{v_i\}}$. It is worthwhile to shrink G to g_i, and explore subgraphs in the branch $T_{\{v_i\}}$ over g_i since (1) g_i is much smaller than G so checking the neighbors of each vertex becomes much faster, and (2) this is an efficient one-time processing that can benefit the entire branch $T_{\{v_i\}}$. In contrast, when processing $T_{\{v_i\}}$, even though whenever a vertex v is added to S, we can shrink g_i further to remove vertices more than c hops away from v, this cost is associated with each expansion of S and so not worthwhile.

We then create branches $T_{\{v_i\}}$ in the degeneracy ordering as shown in Lines 5-6, which keeps the sizes of all g_i's small to avoid having a giant branch that needs to search a deep set-enumeration subtree with potentially high node fanouts. This is because a high-degree vertex v_i tends to appear later in the degeneracy ordering, so V_i has excluded many neighbors of v_i originally in V (see Line 11).

Without loss of generality, assume that $c = 2$, that is, an MkP P has $|P| \geq 2k-1$ (i.e., $|P| > 2k-2$). To create g_i for each v_i, Line 7 first obtains one-hop neighbors of v_i in V_i, and Line 8 then uses them to obtain the two-hop neighbors (where $N(H_1) = \cup_{v \in N(H_1)} N(v)$); finally, Line 9 creates g_i using them (i.e., excluding other vertices in V_i that are > 2 hops away from v_i). Note that it is easy to extend g_i for the general case of c. For example, when $c = 3$, we can additionally compute $H_3 \leftarrow N(H_2) \cap \{v_{i+1}, \ldots, v_n\}$ and then compute g_i to be the subgraph of G induced by $(\{v_i\} \cup H_1 \cup H_2 \cup H_3)$.

Once g_i is created, Line 10 then explores it by extending $S = \{v_i\}$ using branch and bound. For now, we can think of BB(.) simply as BB_basic(.) presented in Algorithm 1, and two variants will be presented for U-MkP and AltRB, respectively. We then remove v_i from G in Line 11 (similar to Line 5 of Algorithm 1) so that later branches will not consider v_i to avoid redundancy. Note that we do not actually need to do the intersections in Lines 7 and 8 since $\{v_1, \ldots, v_{i-1}\}$ have already been removed in previous iterations.

The Reduction Procedure. Recall from Line 2 of Algorithm 1 that BB(.) first calls reduce_and_prune(.) over the instance (S, R, g)

Algorithm 3: Reduction and Pruning Function in AltRB

```
1  function reduce_and_prune(S, R, g)  // returns if T_S is pruned
2      Apply reduction rules to (S, R, g)  // altRB may update ub
3      if (|P| = ub) then return true
4      if g_union = g[S ∪ R] is a k-plex then
5          if |S| + |R| > |P| then P ← S ∪ R
6          return true
7      if no candidate in R is valid then return true
8      return false
```

for further reduction before extending S with vertices from R, the pseudocode of which is shown in Algorithm 3. Specifically, Line 2 first applies the various reduction rules to reduce the candidate size of R. Note that some rules may directly move some candidates in R into S, such as **BR1** and **BR2** in U-MkP [4]. Also, while ub does not change after the preprocessing stage in U-MkP [4], it may be updated the by altRB method of AltRB [14].

..... here ..... U-MkP [4], AltRB [14]

checks if the current P already reaches the maximum possible size ub, and if so, it returns *true* to notify BB(.) to terminate its subgraph exploration (see Line 2 of Algorithm 1). Right after the call of BB(.) in Line 10 of Algorithm 2, we can check if $|P| = ub$ and return P directly if so to terminate early.

Before extending S with vertices in R (e.g., Lines 3–5 in Algorithm 1), we first conduct a look-ahead pruning in Line 4 to see if the entire $g_{\text{union}} = g[S \cup R]$ is a k-plex, and if so, Line 6 returns true so that Line 2 of Algorithm 1 will return directly to terminate its exploration of T_S. Moreover, if $S \cup R$ is larger than the current P, P is updated with $S \cup R$ in Line 5.

Finally, we check if T_S can still produce a k-plex larger than P, and if not, Line 7 returns *true* to let BB(.) terminate its exploration of T_S. The function partition(.) (to be introduced in Section **??**) uses upper-bound-based pruning to return the subset of vertices in R, denoted by B, that is still worth branching with (i.e., to extend S only with a vertex in B in the next step), and if $B = \emptyset$, there is nothing worth extending with so the entire T_S is pruned.

... line 2 and line 7 differ;

4 Set Representations

5 Learning-Based Algorithm Selection

... First summarize algorithm candidates here ...

6 Experiments

7 Conclusion

References

[1] [n. d.]. KONECT. http://konect.cc/networks/.
[2] [n. d.]. Network Repository. https://networkrepository.com.
[3] [n. d.]. SNAP. https://snap.stanford.edu/.
[4] Akhlaque Ahmad, Da Yan, Xiao Chen, Lyuheng Yuan, Qin Zhang, and Saugat Adhikari. 2020. Maximum k-Plex Finding: Choices of Pruning Techniques Matter! *Proc. VLDB Endow.* 14, 9 (2020).
[5] Gary D Bader and Christopher WV Hogue. 2003. An Automated Method for Finding Molecular Complexes in Large Protein Interaction Networks. *BMC bioinformatics* 4, 1 (2003), 2.
[6] Balabhaskar Balasundaram, Sergiy Butenko, and Illya V. Hicks. 2011. Clique Relaxations in Social Network Analysis: The Maximum k-Plex Problem. *Oper. Res.* 59, 1 (2011), 133–142.

[7] Vladimir Batagelj and Matjaz Zaversnik. 2003. An O(m) Algorithm for Cores Decomposition of Networks. *CoRR* cs.DS/0310049 (2003). http://arxiv.org/abs/cs/0310049

[8] Dongbo Bu, Yi Zhao, Lun Cai, Hong Xue, Xiaopeng Zhu, Hongchao Lu, Jingfen Zhang, Shiwei Sun, Lunjiang Ling, Nan Zhang, et al. 2003. Topological Structure Analysis of the Protein-Protein Interaction Network in Budding Yeast. *Nucleic acids research* 31, 9 (2003), 2443–2450.

[9] Lijun Chang, Mouyi Xu, and Darren Strash. 2022. Efficient Maximum k-Plex Computation over Large Sparse Graphs. *Proc. VLDB Endow.* 16, 2 (2022), 127–139.

[10] Lijun Chang and Kai Yao. 2024. Maximum k-Plex Computation: Theory and Practice. *Proc. ACM Manag. Data* 2, 1 (2024), 63:1–63:26.

[11] Alessio Conte, Donatella Firmani, Caterina Mordente, Maurizio Patrignani, and Riccardo Torlone. 2017. Fast Enumeration of Large k-Plexes. In *KDD*. ACM, 115–124.

[12] Alessio Conte, Tiziano De Matteis, Daniele De Sensi, Roberto Grossi, Andrea Marino, and Luca Versari. 2018. D2K: Scalable Community Detection in Massive Networks via Small-Diameter k-Plexes. In *KDD*. ACM, 1272–1281.

[13] Jian Gao, Jiejiang Chen, Minghao Yin, Rong Chen, and Yiyuan Wang. 2018. An Exact Algorithm for Maximum k-Plexes in Massive Graphs. In *IJCAI*. ijcai.org, 1449–1455.

[14] Shuohao Gao, Kaiqiang Yu, Shengxin Liu, and Cheng Long. 2024. Maximum k-Plex Search: An Alternated Reduction-and-Bound Method. *Proc. VLDB Endow.* 18, 2 (2024), 363–376.

[15] John Hopcroft, Omar Khan, Brian Kulis, and Bart Selman. 2004. Tracking Evolving Communities in Large Linked Networks. *Proceedings of the National Academy of Sciences* 101, suppl 1 (2004), 5249–5253.

[16] Haiyan Hu, Xifeng Yan, Yu Huang, Jiawei Han, and Xianghong Jasmine Zhou. 2005. Mining Coherent Dense Subgraphs Across Massive Biological Networks for Functional Discovery. *Bioinformatics* 21, suppl_1 (2005), i213–i221.

[17] Hua Jiang, Fusheng Xu, Zhifei Zheng, Bowen Wang, and Wei Zhou. 2023. A Refined Upper Bound and Inprocessing for the Maximum K-plex Problem. In *IJCAI*. ijcai.org, 5613–5621.

[18] Hua Jiang, Dongming Zhu, Zhichao Xie, Shaowen Yao, and Zhang-Hua Fu. 2021. A New Upper Bound Based on Vertex Partitioning for the Maximum K-plex Problem. In *IJCAI*. ijcai.org, 1689–1696.

[19] John M. Lewis and Mihalis Yannakakis. 1980. The Node-Deletion Problem for Hereditary Properties is NP-Complete. *J. Comput. Syst. Sci.* 20, 2 (1980), 219–230.

[20] Junqiu Li, Xingyuan Wang, and Yaozu Cui. 2014. Uncovering the Overlapping Community Structure of Complex Networks by Maximal Cliques. *Physica A: Statistical Mechanics and its Applications* 415 (2014), 398–406.

[21] Jeffrey Pattillo, Nataly Youssef, and Sergiy Butenko. 2013. On Clique Relaxation Models in Network Analysis. *European Journal of Operational Research* 226, 1 (2013), 9–18.

[22] Stephen B Seidman. 1983. Network structure and minimum degree. *Social networks* 5, 3 (1983), 269–287.

[23] Stephen B Seidman and Brian L Foster. 1978. A Graph-Theoretic Generalization of the Clique Concept. *Journal of Mathematical Sociology* 6, 1 (1978), 139–154.

[24] Duygu Ucar, Sitaram Asur, Umit Catalyurek, and Srinivasan Parthasarathy. 2006. Improving Functional Modularity in Protein-Protein Interactions Graphs using Hub-Induced Subgraphs. In *European Conference on Principles of Data Mining and Knowledge Discovery*. Springer, 371–382.

[25] Zhengren Wang, Yi Zhou, Chunyu Luo, and Mingyu Xiao. 2023. A Fast Maximum k-Plex Algorithm Parameterized by the Degeneracy Gap. In *IJCAI*. ijcai.org, 5648–5656.

[26] Zhengren Wang, Yi Zhou, Mingyu Xiao, and Bakhadyr Khoussainov. 2022. Listing Maximal k-Plexes in Large Real-World Graphs. In *WWW*. ACM, 1517–1527.

[27] Mingyu Xiao, Weibo Lin, Yuanshun Dai, and Yifeng Zeng. 2017. A Fast Algorithm to Compute Maximum k-Plexes in Social Network Analysis. In *AAAI*. AAAI Press, 919–925.

[28] Jiongzhi Zheng, Mingming Jin, Yan Jin, and Kun He. 2023. Two New Upper Bounds for the Maximum k-plex Problem. *CoRR* abs/2301.07300 (2023).

[29] Yi Zhou, Shan Hu, Mingyu Xiao, and Zhang-Hua Fu. 2021. Improving Maximum k-plex Solver via Second-Order Reduction and Graph Color Bounding. In *AAAI*. AAAI Press, 12453–12460.